

Лабораторная №5. Создание основных компонентов образа: дерево устройств, корневая файловая система и загрузчик

Цель работы

Изучить назначение дерева устройств, OpenSBI, U-Boot, конфигурации Extlinux и корневой файловой системы, а затем подготовить эти компоненты для последующей записи образа на носитель.

Оборудование и предварительные требования

- рабочий компьютер с ALT Linux и доступом в интернет;
- установленный кросс-тулчейн RISC-V;
- утилиты `git`, `make` и зависимости для сборки U-Boot и OpenSBI;
- ядро, `.config` и DTB, полученные в лабораторной №4;
- не менее 10 Гбайт свободного места для исходного кода и загруженных компонентов.

Ключевые понятия

Дерево устройств

Определение

Device Tree (DT) — иерархическая структура данных, которая описывает оборудование платформы для ядра операционной системы.

Device Tree позволяет одному ядру работать на разных платах без перекомпиляции при наличии подходящего описания оборудования. Дополнительные сведения приведены в статье на altlinux.org.

Определение

DTS (Device Tree Source) — человекочитаемый исходный файл описания дерева устройств с расширением `.dts` или включаемый фрагмент с расширением `.dtsi`.

Определение

DTB (Device Tree Blob) — бинарное представление дерева устройств, полученное компиляцией DTS и передаваемое ядру при загрузке.

Основным источником DTS служат исходный код Linux и репозитории разработчиков оборудования. В дереве ядра описания обычно находятся в каталоге `arch/архитектура/boot/dts/производитель/`.

Для Lichee RV Dock исходный файл расположен по пути:

```
arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dts
```

После сборки ядра соответствующий DTB находится здесь:

```
arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb
```

Проверка

Практический ориентир. Если готового DTS для платы нет, отправной точкой может служить описание наиболее похожего устройства. Его адаптируют по документации SoC, электрической схеме и распиновке платы.

Цепочка загрузки

Определение

Загрузчик (bootloader) — программа, которая после включения системы подготавливает оборудование и загружает следующий программный компонент или ядро операционной системы.

Для одноплатных компьютеров широко применяется Das U-Boot, обычно называемый U-Boot.

Определение

SPL (Secondary Program Loader) — компактная первая стадия U-Boot, которая выполняет раннюю инициализацию, в том числе настраивает оперативную память для загрузки следующих компонентов.

Определение

OpenSBI — реализация интерфейса Supervisor Binary Interface для RISC-V, которая работает в привилегированном M-mode и предоставляет среде S-mode стандартные низкоуровневые сервисы.

Для Lichee RV Dock цепочку загрузки можно представить так:

1. После подачи питания встроенный BootROM процессора ищет загрузочный код на поддерживаемом носителе и считывает SPL из предусмотренной для него области.
2. SPL выполняет раннюю инициализацию, включая настройку DRAM, и загружает последующие компоненты комбинированного образа U-Boot.
3. OpenSBI запускается в M-mode, остаётся резидентным и передаёт управление U-Boot proper в S-mode.
4. U-Boot proper инициализирует необходимую периферию, читает конфигурацию загрузки и загружает ядро Linux и DTB.
5. U-Boot передаёт ядру управление, а ядро обращается к сервисам OpenSBI через вызовы SBI, например при перезагрузке системы.

BootROM не читает `extlinux.conf` и не ищет ядро в файловой системе: это выполняется на более позднем этапе U-Boot.

Корневая файловая система

Определение

Rootfs (root filesystem) — корневая файловая система, содержащая пользовательское пространство Linux: программы, библиотеки, конфигурацию и каталоги, необходимые после запуска ядра.

После инициализации ядро монтирует rootfs и запускает первый процесс, обычно систему инициализации. В rootfs находятся:

- исполняемые программы в `/bin`, `/sbin` и `/usr/bin`;
- библиотеки в `/lib` и `/usr/lib`;
- конфигурационные файлы в `/etc`;
- файлы устройств в `/dev`;
- временные данные и точки монтирования;
- домашние каталоги пользователей в `/home`.

Без rootfs ядро не сможет запустить пользовательские приложения и системные службы. Подходящий архив можно выбрать на странице регулярных сборок ALT Linux для riscv64. Для этой работы предлагается сборка с Xfce, если преподаватель не указал другой вариант.

Конфигурация Extlinux

Определение

extlinux.conf — текстовый файл конфигурации, который сообщает U-Boot пути к ядру и DTB, а также параметры командной строки ядра.

Простейшая конфигурация для подготовленных компонентов выглядит так:

```
label Linux
    kernel /Image
    fdt /sun20i-d1-lichee-rv-dock.dtb
    append root=/dev/mmcblk0p2 rootwait console=ttyS0,115200
```

Параметры имеют следующее назначение:

- `kernel /Image` задаёт путь к ядру на загрузочном разделе;
- `fdt /sun20i-d1-lichee-rv-dock.dtb` задаёт путь к DTB;
- `root=/dev/mmcblk0p2` указывает раздел с корневой файловой системой;
- `rootwait` предписывает ядру дожидаться появления корневого устройства;

- `console=ttyS0,115200` включает вывод консоли ядра через UART со скоростью 115200 бод.

Внимание

Значения `kernel`, `fdt` и `root` должны соответствовать фактическим путям и разметке будущего носителя. Ошибка в любом из них остановит загрузку системы.

Порядок выполнения

Получение DTB

Возьмите DTB Lichee RV Dock из результатов лабораторной №4 и убедитесь, что имя платы определено верно:

```
file arch/riscv/boot/dts/allwinner/sun20i-d1-lichee-rv-dock.dtb
```

Проверка

Ожидаемый результат. Утилита `file` должна определить файл как Flattened Device Tree blob, а не как пустой файл или обычный текст.

Сборка OpenSBI

На сайте сообщества `linux-sunxi` приведён вариант сборки компонентов для Lichee RV. Получите исходный код OpenSBI и соберите динамическую прошивку:

```
git clone https://github.com/riscv-software-src/opensbi
cd opensbi
make CROSS_COMPILE=riscv64-linux-gnu- PLATFORM=generic FW_PIC=y
cd ..
```

Результат сборки, используемый U-Boot, находится по пути:

```
opensbi/build/platform/generic/firmware/fw_dynamic.bin
```

Сборка U-Boot

Получите ветку U-Boot для Allwinner D1, создайте конфигурацию Lichee RV Dock и запустите сборку, указав путь к OpenSBI:

```
git clone https://github.com/smaeul/u-boot -b d1-wip
cd u-boot
make CROSS_COMPILE=riscv64-linux-gnu- lichee_rv_dock_defconfig
make CROSS_COMPILE=riscv64-linux-gnu-
  ↪ OPENSBI=../opensbi/build/platform/generic/firmware/fw_dynamic.bin
cd ..
```

После успешной сборки в каталоге `u-boot` появится комбинированный образ `u-boot-sunxi-with-spl.bin`.

Проверка

Проверка сборки. Убедитесь, что `u-boot-sunxi-with-spl.bin` существует и имеет ненулевой размер. Сохраните журнал сборки для отчёта.

Получение готового U-Boot из репозитория

Вместо самостоятельной сборки можно использовать готовый U-Boot с OpenSBI из репозитория ALT Linux. Для сравнения со самостоятельно собранным вариантом получите пакет `u-boot-sunxi-riscv`.

Если репозиторий `sisyphus_riscv64` не настроен, добавьте следующие строки в `/etc/apt/sources.list`:

Опасность повреждения

Полная замена `/etc/apt/sources.list` уничтожит текущий список источников пакетов. Сначала сохраните резервную копию файла и после получения пакета восстановите исходную конфигурацию.

```
rpm [sisyphus-riscv64]
↪ http://ftp.altlinux.org/pub/distributions/ALTLinux/ports/riscv64
↪ Sisyphus/riscv64 classic
rpm [sisyphus-riscv64]
↪ http://ftp.altlinux.org/pub/distributions/ALTLinux/ports/riscv64
↪ Sisyphus/noarch classic
```

Обновите индекс и установите пакет:

```
apt-get update
apt-get install u-boot-sunxi-riscv
```

Файлы загрузчиков для устройств на базе Allwinner D1 будут установлены в `/usr/share/u-boot`. Найдите вариант для Lichee RV Dock и сравните его размер и контрольную сумму с самостоятельно собранным образом:

```
sha256sum u-boot/u-boot-sunxi-with-spl.bin
```

Совпадение контрольных сумм не требуется: версии исходного кода и параметры сборки могут различаться.

Подготовка `extlinux.conf`

Создайте конфигурацию на основе приведённого примера. Укажите согласованную с преподавателем метку, по которой можно определить автора будущего образа, и проверьте пути к ядру, DTB и rootfs.

Проверка

Проверка конфигурации. В файле должны присутствовать директивы `label`, `kernel`, `fdt` и `append`; значения путей должны совпадать с именами подготовленных файлов и будущей разметкой носителя.

Получение rootfs

Скачайте указанный преподавателем архив `rootfs` или предложенную регулярную сборку ALT Linux для `riscv64`. Не распаковывайте архив без необходимости на этом этапе, но проверьте его тип, размер и контрольную сумму.

Внимание

Сохраните ядро, DTB, загрузчик, `extlinux.conf` и архив `rootfs`. Все эти компоненты потребуются при создании образа в следующей лабораторной работе.

Задание

1. Получите DTB Lichee RV Dock из результатов сборки ядра и проверьте тип файла.
2. Соберите OpenSBI и U-Boot для Lichee RV Dock.
3. Получите аналогичный готовый загрузчик из репозитория Sisyphus-riscv64.
4. Сравните размеры и контрольные суммы самостоятельно собранного и пакетного загрузчиков.
5. Скачайте с указанного преподавателем ресурса архив `rootfs`.
6. Подготовьте `extlinux.conf`, указав согласованную метку автора будущего образа.
7. Сохраните ядро, DTB, оба варианта загрузчика, `extlinux.conf` и архив `rootfs`.
8. Прдемонстрируйте преподавателю подготовленные компоненты.

Контрольные вопросы

1. Что такое Device Tree и какую роль оно выполняет при загрузке Linux?
2. Чем файл `.dts` отличается от `.dtb`?
3. Для чего нужны SPL и U-Boot proper в цепочке загрузки?
4. Какую роль выполняет OpenSBI при загрузке RISC-V?
5. Какая информация содержится в файле `extlinux.conf`?

Требования к отчёту

Отчёт должен содержать:

- цель работы;
- схему цепочки загрузки Lichee RV Dock;
- путь и результат проверки DTB;
- команды и результаты сборки OpenSBI и U-Boot;
- сведения о пакетном U-Boot и сравнение двух вариантов;
- содержимое подготовленного `extlinux.conf`;
- имя, размер и контрольную сумму архива `rootfs`;
- перечень сохранённых компонентов;
- ответы на контрольные вопросы;
- краткий вывод.

Подготовьте отчёт в согласованном с преподавателем формате и отправьте его до установленного срока.